Object Oriented Programming (OOP)

Semester: 2nd
Credit Hour: (01)
Department of Computer Science
Course Instructor: Engr. Zakria Bacha

Lab Task No. 7

Class & Object in C++

Previous Lab:

In previous lab, we have covered below three topics:

1. **What is Class**
2. **What is Object of a Class**
3. **Methods in Class**
4. **Making multiple files for OOP project**

Current Topics for Lab:

1. **C++ Constructors**
2. **Constructor with Parameters**
3. **Constructor Defined Outside the Class**
4. **Methods in Class**
5. **Constructor Overloading**
6. **Constructor Chaining**
7. **Types of Constructors in C++**

Constructors

A constructor is a special method that is automatically called when an object of a class is created.

Create a Constructor: To create a constructor, use the same name as the class, followed by parentheses ():

```
class MyClass {      // class
  public:            // Access specifier

    MyClass()
    {     // Constructor
      cout << "I am constructor ";
    }
};

int main() {
  MyClass myObj;     // Create an object of MyClass
                     //(this will call the constructor)
```

Figure 1

Constructor Rules

1. The constructor has the **same name as the class**.
2. It has **no return type** (not even `void`).
3. It is usually declared **public**.
4. It is **automatically called** when an object is created.

Constructor with Parameters:

1. Constructors can also take parameters (just like regular functions), which can be useful for setting initial values for attributes.

2. When we call the constructor (by creating an object of the class), we pass parameters to the constructor, which will set the value of the corresponding attributes to the same:

```
class Car {          // class
  public:            // Access specifier
    string brand;    // Attribute
    string model;    // Attribute
    int year;        // Attribute
    Car(string x, string y, int z)
    {                                  // Constructor
                                       //with parameters
      brand = x;
      model = y;
      year = z;
    }
};
```

```
int main() {
  // Create Car objects and call the constructor
  // with different values
  Car carObj1("BMW", "X5", 1999);
  Car carObj2("Ford", "Mustang", 1969);
```

Constructor Defined Outside the Class:

You can also define the constructor outside the class using the scope resolution operator ::

```
class Car {          // The class
  public:            // Access specifier
    string brand;    // Attribute
    string model;    // Attribute
    int year;        // Attribute
    Car(string x, string y, int z); // Constructor declaration
};

// Constructor definition outside the class
Car::Car(string x, string y, int z) {
  brand = x;
  model = y;
  year = z;
}
```

Constructor Overloading

- In C++, you can have more than one constructor in the same class. This is called constructor overloading.
- Each constructor must have a different number or type of parameters, so the compiler knows which one to use when you create an object.

Why Use Constructor Overloading?

1. To give flexibility when creating objects
   a. Constructor overloading allows objects to be created in ***different ways depending on the available information.***
2. To set default or custom values
   a. One constructor can assign **default values**, while another allows **user-provided values**.
3. To reduce repetitive code
   a. Constructors automatically **initialize object data**, which reduces repeated code and keeps the program cleaner.

Example with Two Constructors:

This class has two constructors: one without parameters, and one with parameters:

```
1  #include<iostream>
2  using namespace std;
3  class Car {
4    public:
5      string brand;
6      string model;
7
8      Car() {
9        brand = "Unknown";
10       model = "Unknown";
11     }
12
13     Car(string b, string m) {
14       brand = b;
15       model = m;
16     }
17 };
```

Figure 2

```
19 int main() {
20   Car car1;
21   Car car2("BMW", "X5");
22   Car car3("Ford", "Mustang");
23
```

You cannot call multiple constructors sequentially for a single object after it has been created

Calling One Constructor from Another (Constructor Chaining)

```
class Student {
public:
    Student() {
        cout << "Default Constructor" << endl;
    }

    Student(int id) : Student() {   // calls default constructor
        cout << "Constructor with ID: " << id << endl;
    }
};

int main() {
    Student s1(10);
}
```

Types of Constructors in C++

Constructors can be classified based on the situations they are being used in. There are 4 types of constructors in C++:

1. ***Default Constructor (Already covered)***
2. ***Parameterized Constructor (Already covered)***
3. ***Copy Constructor***
4. ***Move Constructor***

Default Constructor (Already covered)

1. If a class has **no constructor**, the compiler automatically creates a **default constructor**.
2. The **default constructor takes no parameters** and initializes the object with default values.
3. But if the **programmer defines any constructor (user-defined)**, the compiler **does not create the default constructor**.

```
class Student {
public:
    int id;

    Student(int x) {    // parameterized constructor
        id = x;
    }
};

int main() {
    Student s1;
}
```

Correct or not?

Parameterized Constructor:

A parameterized constructor lets us pass arguments to initialize an object's members. It is created by adding parameters to the constructor and using them to set the values of the data members. (Already covered)

Copy Constructor:

A copy constructor is a member function that initializes an object using another object of the same class. Copy constructor takes a reference to an object of the same class as an argument.

```
class A {
public:
    int val;
    // Parameterized constructor
    A(int x) {
        val = x;
    }
    // Copy constructor
    A(A& a) {
        val = a.val;
    }
};
int main() {
    A a1(20);
    // Creating another object from a1
    A a2(a1);

    cout << a2.val;
    return 0;
}
```

National University of Computer and Emerging Sciences

Peshawar Campus

Lab Task No. 1: Constructor overloading

Create a class named Student that stores the student's ***name, roll number, and marks***. In this task, you will implement ***constructor overloading.*** First, create a ***default constructor*** that initializes the data members with some basic values. Then create a ***parameterized constructor*** that allows the user to provide values for name, roll number, and marks at the time of object creation. In the main() function, create multiple objects of the Student class using both constructors. Display the student information to observe how different constructors initialize the objects differently.

==========

==========

Task 2: Multiple Constructors

Design a class named Result that represents a simple student result management system. The class should contain the following data members***: student name, roll number, and marks of five subjects***. ***Implement multiple constructors for this class***. One constructor should initialize the student information with default values, while another constructor should allow the user to provide the student name, roll number, and subject marks during object creation. Create methods to ***calculate the average marks and display the complete result***. In the main() function, create two student objects using different constructors and display their marks, average, and pass/fail status. A student is considered ***pass if the average marks are 50 or above.***

==========

==========

Task 3: Method Inside Class

Create a class named Employee to manage employee salary information. The class should contain the following data members: employee name, employee ID, and basic salary. Declare the class in a header file and define the member functions in a separate .cpp file using the scope resolution operator (::).

Implement a constructor to initialize employee details and create functions to calculate a 10% bonus based on the salary and display the employee information. Organize the program into three separate files: class_header.h, class_header.cpp, and main.cpp. In the main() function, create multiple employee objects and display their basic salary, bonus amount, and final salary after adding the bonus.

==========

==========